AI INSTITUTE #AIISC
UNIVERSITY OF SOUTH CAROLINA

UNIVERSITY OF SOUTH CAROLINA

*CSCE 190: Computing in the Modern World*

# Class 7 – Testing (Solving with Computing)

PROF. BIPLAV SRIVASTAVA, AI INSTITUTE

OCT 2026

**Carolinian Creed: "I will practice personal and academic integrity."**

**Credits**: **Copyrights of all material reused acknowledged**

# Outline

- Introduction Section
  - Recap from Last Class
- Main Section
  - Problem Solving Process
  - **Validating the Problem**
    - **Testing – Concepts and Activities**
  - Selecting Problems that Matter
- Conclusion Section
  - Ask Me Anything

# Introduction Section

# Recap from Last Lecture - Class 7

We talked about

- Problem Solving Process
- **Validating the Problem**
  - **Creating Requirements and Specifications**
- Selecting Problems that Matter

# Assignment 7: Same for All in a Team

- Group 4-5 students
- Discuss a problem category
- Decide on a problem – see ***problems that matter***
- Write a problem statement
  - The group will have one problem statement
  - Add your statement to your GitHub in a sub-folder called ‘assignment-6’
    - Do mention all the students in your group

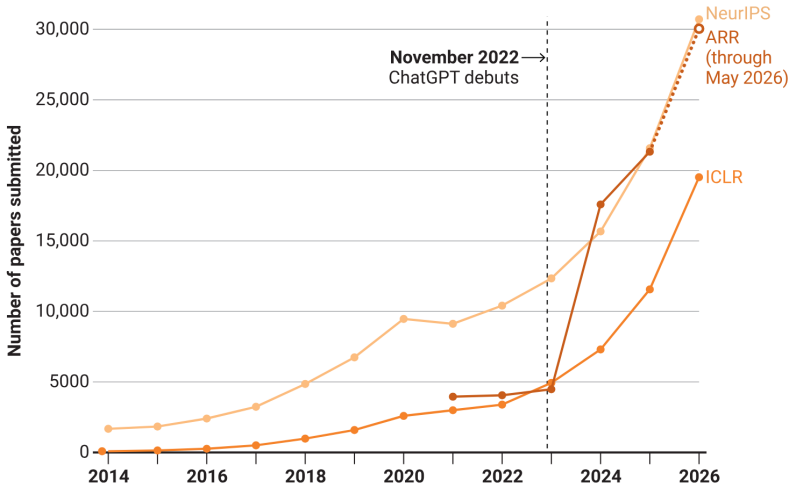
- Write for **client**
  - **Requirements**: what will meet the need?
- Write for **developers**
  - **Specification**: what will the system-to-be built do
  - **Test plan**: how will we show that the built system works to meet the specification
- Add requirements, specification, and test plan to your Github in a sub-folder called ‘assignment-7’
  - Do mention all the students in your group

# Computing in the News

AI slop tsunami, Oct 2026, https://www.science.org/content/article/ai-triggers-tsunami-submissions-computer-science-conferences-impose-caps

Arxiv received 40,363 submissions in September 2026, nearly double the tally of 20,569 in September 2024

[Research ecosystem is overloaded, while 70-80% papers are non-reproducible]




Conference on Neural Information Processing Systems (NeurIPS); International Conference on Learning Representations (ICLR)

(GRAPHIC) V. PENNEY/*SCIENCE*; (DATA) ASSOCIATION FOR COMPUTATIONAL LINGUISTICS ROLLING REVIEW; CONFERENCE ON NEURAL INFORMATION PROCESSING SYSTEM; INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS; OPENACCEPT.ORG.

# Main Section

# Problem Solving Process

**Receiving** the problem ***{from client}***

- Client creates **requirements** document

- **Understanding the problem and scoping**
  - Developer creates **specification**

- **Solving**
  - Understand and implementing with tools for solving the problem: **Design docs, test plans**
  - Create solutions: **Code, Release**
  - Evaluate solutions: **Test logs**

- **Delivering** ***{to customers}***

A generic process involves:

- Customer:
  - requirements
- Developer:
  - specification
  - design
  - development (coding)
  - testing
  - release
- Customer:
  - evaluation
- acceptance

The above process is highly simplified and rarely happens in one-round. Read about 'Software Development Life Cycles' (SDLCs).

# Validating the Problem

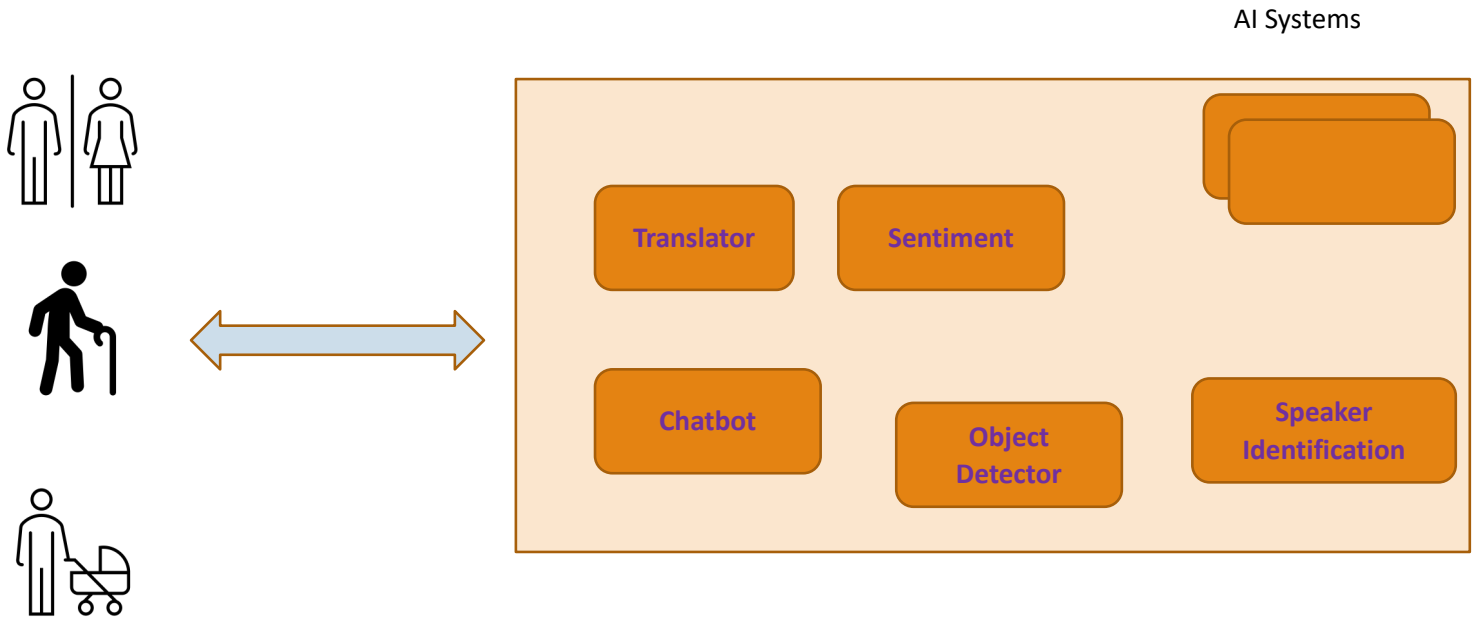
- **Objective**: Understanding the problem and scoping
- Think and articulate – what will be
  - The inputs: e.g. data, current process
  - The outputs (deliverables): e.g., software, solution, documentation
  - Criteria for problem being solved: e.g., metrics

  - Considerations
    - Current practices/ processes
    - Why would someone use your solution to the problem?
    - **If you can't identify inputs, outputs, and evaluation metrics, the problem is most likely ill-defined computationally. Needs more work!**

# Selecting Problems that Matter

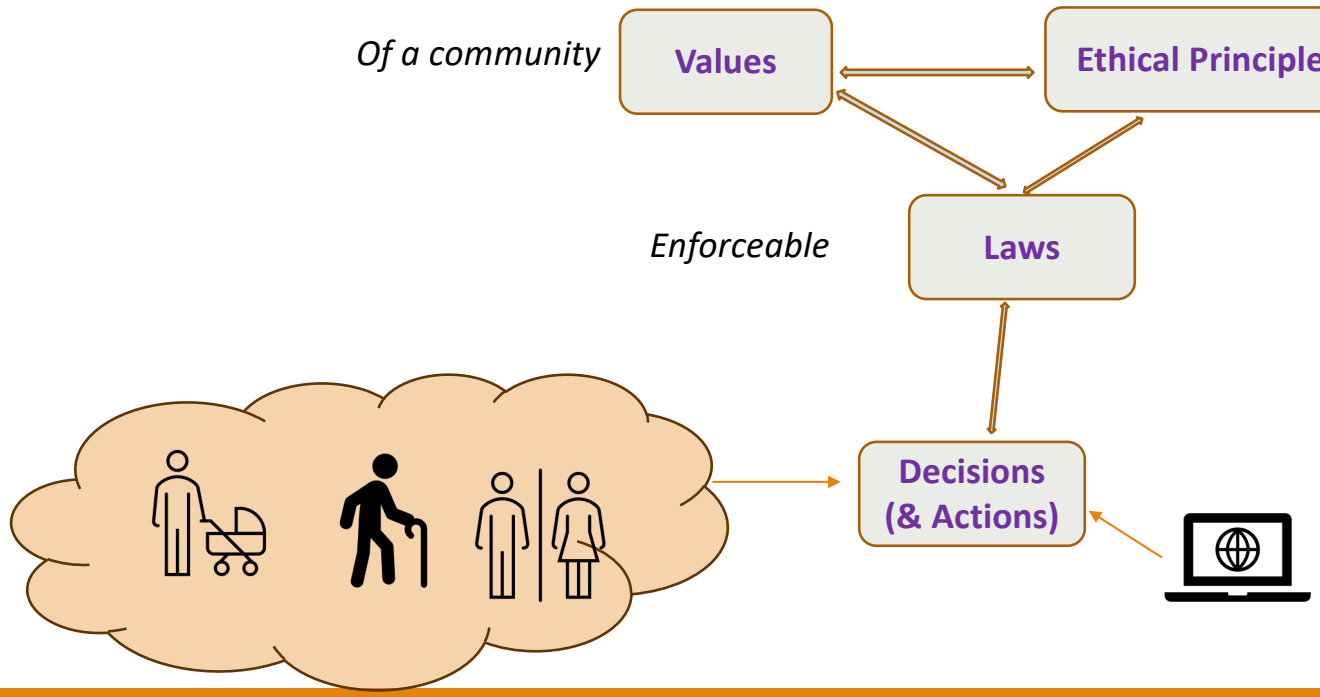
- Answer
  - For whom is the problem important? **[A lot of people]**
  - What happens if you solve it well? **[With change the status quo for the better]**
  - Do you have the inputs to get started? **[Yes? Otherwise, re-think problem or inputs]**
  - Do you have the resources/ are you best qualified to solve the problem? Do you need to augment the team? **[Yes? Otherwise, pass or get help]**

# Technology and People




**Trust:** *Can people trust* ***new system*** *to perform capably, consistently, and with human values?*

# Values, Ethics, Law, Decisions (Actions) with Trust




**Trust:** *how can, humans or machines (AI) or in collaboration together, take decisions and actions following the same **values**, **ethical** principles and **laws ?***

**Assumption:** *they should follow same values, principles and laws.*

# Concept – Verification, Validation




A generic process involves:

- Customer:
  - requirements
- Developer:
  - specification
  - design
  - development (coding)
  - testing
  - release
- Customer:
  - evaluation
  - acceptance

The above process is highly simplified and rarely happens in one-round. Read about 'Software Development Life Cycles' (SDLCs).

# Concept – Functional v/s Non-functional

- Requirements – what the customer says they want
  - **Functional**: must have – typically, capabilities
  - **Non-functional**: good to have but may or may not be specified – typically, throughput, reliability,
- Specifications – what the developers say they implemented
  - **Functional**: must have – typically, capabilities
  - **Non-functional**: good to have but may or may not be specified – typically, throughput, reliability,

A generic process involves:

- Customer:
  - requirements
- Developer:
  - specification
  - design
  - development (coding)
  - testing
  - release
- Customer:
  - evaluation
  - acceptance

The above process is highly simplified and rarely happens in one-round. Read about 'Software Development Life Cycles' (SDLCs).

# Concept – Testing Types

- Unit testing – single developers, testing capability implemented
- System and integration testing – by a group of developers, testing system/ sub-system capabilities
- Acceptance testing – overall system's acceptance, **by the customer**

A generic process involves:

- Customer:
  - requirements
- Developer:
  - specification
  - design
  - development (coding)
  - testing
  - release
- Customer:
  - evaluation
  - acceptance

The above process is highly simplified and rarely happens in one-round. Read about 'Software Development Life Cycles' (SDLCs).

# Testing (AI) Chatbots – As Software

- Requirements – what the customer says they want
  - **Functional**: must have – typically, capabilities
  - **Non-functional**: good to have but may or may not be specified – typically, throughput, reliability,
- Specifications – what the developers say they implemented
- Testing types
  - Unit testing – single developers, testing capability implemented
  - System and integration testing – by a group of developers, testing system/ sub-system capabilities
  - Acceptance testing – overall system's acceptance, **by the customer**

# Types of (AI) Software Evaluation

- **Technical evaluation**: is the (new) method accurate?
  - Concern is bug-free implementation
- **Business evaluation**: is the (new) method beneficial ?
  - Usually done in a small time horizon
  - Other factors may correlate with success or failure
- **Causal evaluation**: did the (new) method really work?
  - No other factor but this method contributed to business success

# Exercise – 1 of 2

- Calculate 2 + 3 * 4 – 5 / 6  and express steps in English
  - Explain how you did it
- Calculate 2 + 3 * 4 – 5 / 6  with a calculator
  - Explain how the calculator did it

# Exercise – 2 of 2

- Calculate 2 + 3 * 4 – 5 / 6 and express steps in English
- Explain which LLMs do correctly and how did you infer that

Llama-3-70b-Groq, vis Poe

I'd be happy to help with the calculation.

To evaluate the expression 2 + 3 * 4 – 5 / 6, we need to follow the order of operations (PEMDAS):

Multiply 3 and 4: 3 * 4 = 12
Divide 5 by 6: 5 / 6 = 0.83 (approximately)
Add 2 and 12: 2 + 12 = 14
Subtract 0.83 from 14: 14 - 0.83 = 13.17

Therefore, the final answer is approximately 13.17.

Bot image for Deepseek-V3-FW

Deepseek-V3-FW
Poe

Certainly! Let's solve the expression step by step following the order of operations (PEMDAS/BODMAS: Parentheses/Brackets, Exponents/Orders, Multiplication and Division (left to right), Addition and Subtraction (left to right)).
Expression:

2 + 3 * 4 – 5 / 6

Multiplication (3 * 4):
Multiply 3 and 4 → 12.
Now: 2 + 12 − 5 / 6.

Division (5 / 6):
Divide 5 by 6 → ~0.833 (repeating).
Now: 2 + 12 − 0.833.

Addition (2 + 12):
Add 2 and 12 → 14.
Now: 14 − 0.833.

Subtraction (14 − 0.833):
Subtract 0.833 from 14 → ~13.167.

Final Answer:

≈ 13.167

(Note: Exact value is 14−56=846−56=796≈13.16714−65=684−65=679≈13.167).

Steps are summarized:

Multiply 3 × 4 = 12.
Divide 5 ÷ 6 ≈ 0.833.
Add 2 + 12 = 14.
Subtract 14 − 0.833 ≈ 13.167.

# Problem Example 1: Student Meals Reco

- **Problem**: build a system to help students choose meals during study days
- Validating the problem:
  - The inputs: e.g. data, current process
  - The outputs (deliverables): e.g., software (app)
  - Criteria for problem being solved: e.g., adoption rate
- Solving the problem
  - Recommendation methods
- Trustworthiness ecosystem
  - Values, ethical principles, laws
  - Users impacted

Problem statement criteria

- Are focused on the user (avoid using “we” or “the product”)
- Are broad (leave room for innovation and alternatives)
- Are manageable (just this one problem)

# Problem Example 2: Nobel Prizes

- **Problem**: build a system to contextualize about Nobel Prizes wins
- Validating the problem:
  - The inputs: e.g. data, current process
  - The outputs (deliverables): e.g., software (app)
  - Criteria for problem being solved: e.g., adoption rate
- Solving the problem
  - **Identifying topics and trends**
- Trustworthiness ecosystem
  - Values, ethical principles, laws -
  - Users impacted – ***students making future decisions based on trends ?***

Problem statement criteria

- Are focused on the user (avoid using "we" or "the product")
- Are broad (leave room for innovation and alternatives)
- Are manageable (just this one problem)

# Nobel Prizes: Non-functional Requirements/ Specification

- Winners background
  - Name, gender, age, country, institution
  - Should or shouldn't be given ? Any disclaimer
- What do the awards in a year tell about
  - The past ?
  - The future ?
- Themes
  - Terminology changes – how to handle them

# Snapshot from Different Sections

| Section 1 | Section 2 | Section 4 |
|---|---|---|
| 1. students on scooters (harm)<br>2. Students managing money in college<br>3. Students finding housing (off-campus)<br>4. Students managing assignments<br>**5. (all) Sidewalks uneven**<br>6. Students managing finances – 1st year<br>7. Students: saving meal swipes (7!) | **1. Students: textbook overprices and requirements**<br>2. (All) drunk driving at 5-points<br>**3. Students: on-campus laundry**<br>4. Students: USC bus trackers are unreliable<br>5. Students: non-reserved parking<br>6. Students: double-deck parking | 1. Students: pedestrian safety, accessibility<br>2. Students: campus parking and transportation<br>3. {} – problem not noted<br>4. Students: college tuition payment<br>**5. Students: time management and resources** |

# Assignment 8: Same for All in a Team

- Group 4-5 students
- Discuss a problem category
- Decide on a problem – see ***problems that matter***
- Write a problem statement
  - The group will have one testing plan
  - Add your statement to your GitHub in a sub-folder called 'assignment-8'
    - Do mention all the students in your group

- Work with your group
  - If you have a group, sit with them
  - If you don't, join an existing group.
- Discuss a system setup
  - Inputs and outputs
  - Metrics
- Discuss **evaluation**
  - Functional specification
  - Non-functional specification
- Write test plans
  - A test cases has a sample inputs and expected outputs
  - Q: How many testcases to create ?

**See sample testcases at:** https://github.com/biplav-s/course-cmw-f26/tree/main/software-dev

# Concluding Section

# Conclusion

We talked about

- Problem Solving Process
- Validating the Problem
- Selecting Problems that Matter

| Week | Content | Assignment |
|---|---|---|
| 1 | Intro, syllabus, university resources | Slack account |
| 2 | CSE majors, career paths, research areas | Write a mission statement |
| 3 | Github (account and basics) + Lab | Github account, repo |
| 4 | HTML basics, IDE (VS Code) | None |
| 5 | More HTML, CSS, Problem Solving | Update Github with CSS |
| 6 | Brainstorming, problem statement | Problem statement |
| 7 | Computing – (Software) Dev. Life Cycle | Draft Requirement, Specification |
| 8 | Computing – Testing a System | Test Plan |
| 9 | AI – LLMs/FMs, Generative AI, Issues and Challenges | Problem Solving with GenAI |
| 10 | Building a resume, LinkedIn | Draft resume in PDF |
| 11 | Review resumes | Review resumes, update Github |
| 12 | Convert resume to HTML | Begin conversion, update Github |
| 13 | CSE research areas | (Finalize website) |
| N/A | Last day of classes: 4 Dec, 2026 | **All extensions end** |

# Ask Me Anything